Memorandum

To: Charlie Refvem, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

From: Lucas Nathan
lrnathan@calpoly.edu

Vandan Patel
vpatel43@calpoly.edu

Oliver Lenoir
olenoir@calpoly

Date: 03 March 2026

RE: **Lab 0x06 - State Estimation**

Overview:

The objective of Lab 0x06 was to implement a linear state estimator for the Romi mobile robot that combines a physics-based state-space model with real sensor measurements to improve estimation of the robot's internal states. Rather than relying solely on encoder readings or IMU data independently, the estimator fuses model predictions with measured outputs to produce smoother, more physically consistent estimates. The four states selected for estimation were left wheel displacement, right wheel displacement, heading angle, and heading rate. These states represent the core linear dynamics of Romi and are sufficient to reconstruct the robot's motion. Although global position was not directly included in the linear observer model, it was later reconstructed using the estimated wheel motion and heading.

The first component of this lab was the implementation of a hardware driver for the BNO055 IMU. The driver communicates with the sensor over I2C in controller mode and verifies the chip ID on startup to ensure proper initialization. The IMU was placed into IMU-Plus fusion mode, which uses the accelerometer and gyroscope but excludes the magnetometer. This configuration was selected to reduce heading instability caused by magnetic disturbances while still providing fused orientation estimates. The driver includes methods to change operating mode, retrieve calibration status, read and write calibration coefficients, read Euler angles, and read angular velocity. The heading angle is read in degrees and the yaw rate in degrees per second, and both are converted to radians and radians per second, respectively, before being used in the state estimator to maintain consistency with the state-space model. Calibration status is monitored through the calibration status register, and calibration coefficients can be saved and reloaded to avoid repeated manual calibration. Care was taken to ensure correct sign conventions between the IMU coordinate system and the robot model.

The observer was designed using the linear state-space model developed previously in HW 0x03, simplified to remove nonlinear global position dynamics. The input vector consisted of the

applied left and right motor voltages, computed from PWM effort and battery voltage. The output vector consisted of measurable quantities: left and right wheel displacements from the encoders and heading and yaw rate from the IMU. The observer followed the standard linear form, where the modified input vector $\tilde{u}$ includes both the physical inputs and the measured outputs. The observer gain matrix was selected in continuous time using pole placement, with observer poles placed further left than the system poles to ensure faster convergence of estimation error. The continuous-time model was then converted to discrete time using a sample period of 20 ms, matching the period of the estimator task.

The estimator was implemented as a dedicated task within the cooperative scheduler. The system uses a priority-based scheduling structure, and the estimator task runs at a fixed period of 20 ms with a higher priority than the motor tasks to ensure timely updates. Each update cycle reads the motor voltage shares, encoder displacement shares (converted to meters), and IMU measurements. Encoder counts are converted to linear displacement using wheel radius and counts-per-revolution parameters. Heading measurements are unwrapped to prevent discontinuities at $\pm\pi$ and converted from degrees to radians. Yaw rate measurements are converted from degrees per second to radians per second. After computing the observer update using matrix multiplication, the updated state estimates are published to shared variables for use by other tasks.

During testing, the estimated wheel displacements closely tracked encoder measurements while exhibiting smoother derivatives when used to compute velocity. Raw encoder velocity, obtained by differentiating position, showed noticeable high-frequency noise, especially at low speeds. The observer significantly reduced this noise while preserving dynamic response. When the robot was stationary, estimated wheel velocities converged to zero without drift, indicating proper observer stability and appropriate pole placement. During straight-line motion, the estimated heading closely followed IMU measurements but with reduced jitter due to model-based smoothing. During turning maneuvers, the heading estimate converged rapidly without oscillation. When initial conditions were intentionally offset, the estimator converged to the correct state within a short time window, demonstrating effective error dynamics governed by the chosen observer gains. Increasing the observer gains improved convergence speed but slightly amplified measurement noise, illustrating the tradeoff between responsiveness and noise sensitivity.

Global position was reconstructed using nonlinear kinematic equations applied to the estimated states. The average of the estimated left and right wheel velocities was used to compute forward velocity, and heading was used to resolve motion into x and y components. These were numerically integrated at each time step to estimate global position. For short straight-line motions, the predicted position closely matched expected behavior. However, during extended curved motion, small biases in heading estimation produced gradual drift in global position. This behavior is expected because global position is not directly corrected by measurement feedback in the linear observer and depends entirely on accurate velocity and heading estimation. The results highlight the limitation of purely model-based position tracking without absolute position sensors.

Special attention was given to unit consistency and sign conventions, as incorrect scaling or orientation mismatches can destabilize an observer. All state variables and measurements were

converted into SI units before being used in the observer equations. Encoder displacements were converted to meters, heading to radians, yaw rate to radians per second, and motor effort to applied voltage. These conversions ensured compatibility with the discretized state-space matrices derived offline.

Overall, the state estimator functioned as intended and significantly improved the quality of internal state estimates compared to raw sensor measurements alone. By combining the system model with encoder and IMU feedback, the observer produced smoother, more physically consistent estimates of wheel displacement, heading, and angular velocity. Although long-term global position drift remains unavoidable without additional absolute sensors, the estimator successfully demonstrated the effectiveness of model-based sensor fusion for mobile robotics applications and validated the theoretical observer design presented in lecture.

Appendix A. Diagrams

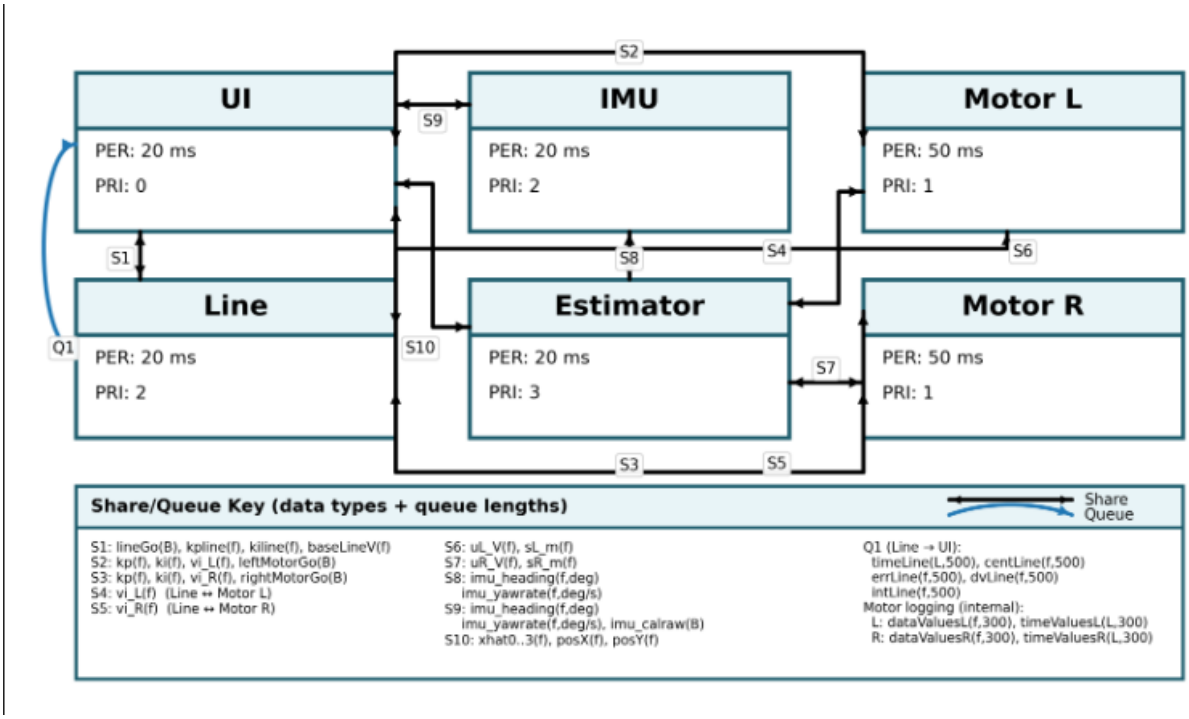

Figure A-1. Task Diagram. Shows how the 6 tasks communicate with each other and the shares and queues.

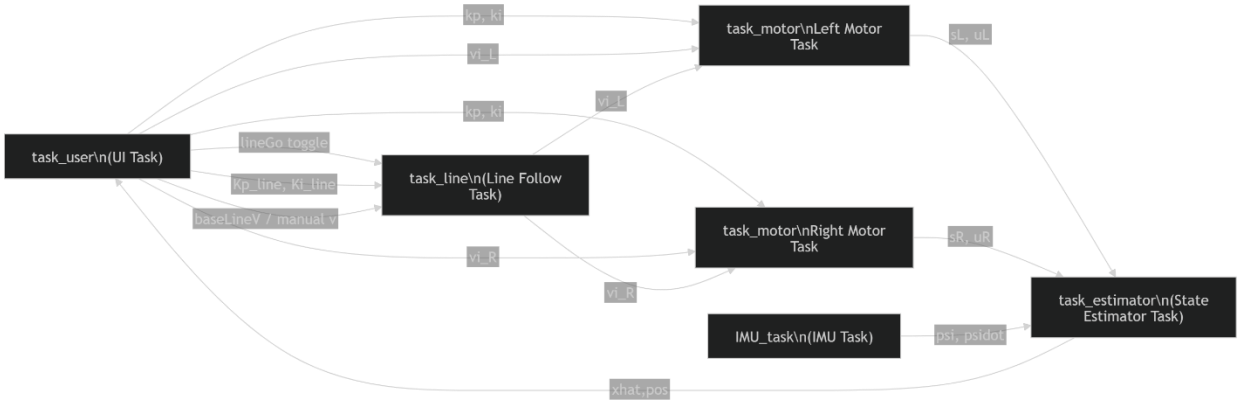

Figure A-2. Updated intertask communication diagram

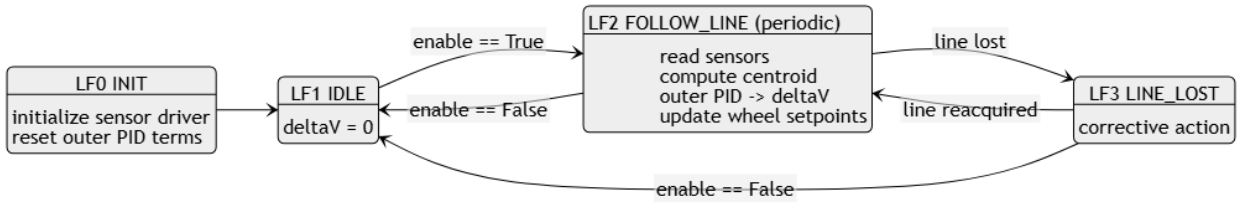

Figure A-3. Line follow finite state machine from Lab 0x05

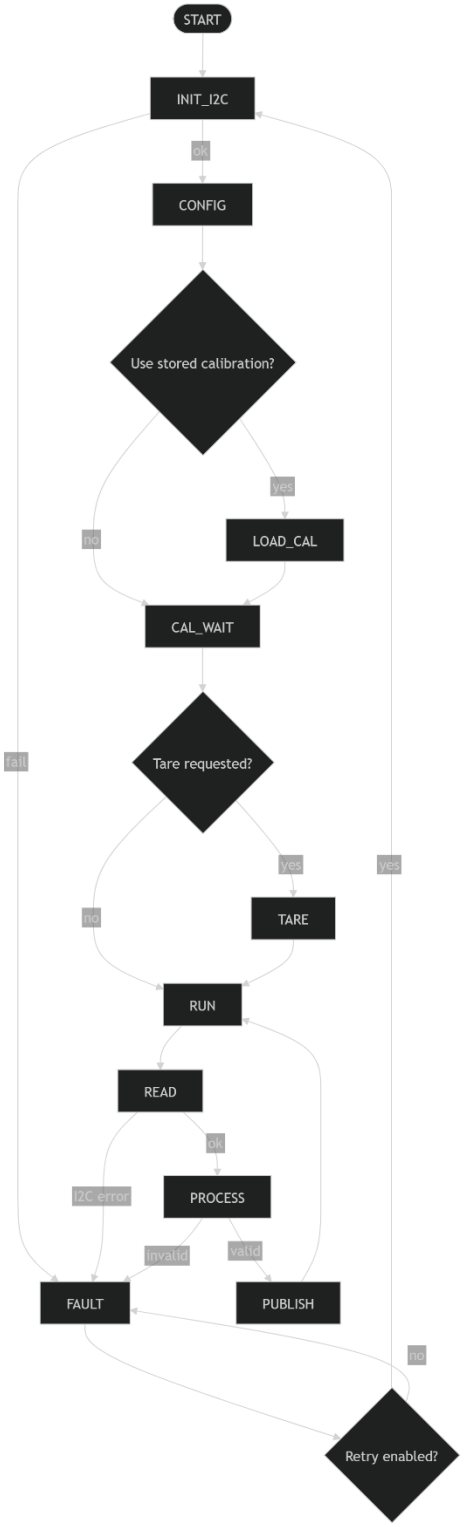

Figure A-4. IMU finite state machine

Appendix C. Source Code

IMU_driver.py

This file implements the hardware driver for the IMU. It handles I2C communication, verifies device initialization, sets the operating fusion mode, and provides methods to read heading, angular velocity, and calibration status. It abstracts low-level register access so that higher-level tasks can retrieve orientation data in a clean and structured manner.

IMU_task.py

This file contains the IMU data acquisition task. It periodically reads heading and yaw rate from the IMU driver and publishes these values to shared variables for use by the state estimator and user interface. It ensures consistent timing of sensor updates within the cooperative scheduler.

task_estimator.py

This file implements the discrete-time state estimator (observer) task. It reads motor voltages, wheel displacement measurements, and IMU data from shared variables, performs the matrix-based observer update, and publishes the estimated states. It also reconstructs global position using the estimated motion states. This task represents the core model-based sensor fusion implementation for Lab 0x06.

matrices.py

This file stores the precomputed discrete-time observer matrices used by the estimator task. These matrices were generated offline using continuous-to-discrete conversion and embedded into firmware to reduce computational load on the microcontroller.

Matrix calculations.py

This file contains the offline Python script used to compute the observer gain and perform continuous-to-discrete conversion of the system matrices. It uses scientific computing tools to generate the discrete matrices that are later copied into the embedded firmware.

task_motor.py

This file was modified to publish additional shared variables required by the state estimator. In addition to performing closed-loop wheel velocity control, it now computes and shares applied motor voltage and wheel displacement in meters. These outputs serve as inputs to the observer model.

Main.py

This file was updated to integrate the IMU driver, IMU task, and state estimator task into the system. It initializes new shared variables for sensor measurements and estimated states, creates the estimator task with appropriate priority and period, and adds it to the cooperative scheduler. It now coordinates both control and state estimation functionality.